Stack
Home: https://docs.haskellstack.org/en/stable/
Glossary: https://docs.haskellstack.org/en/stable/glossary/

HPack: https://github.com/sol/hpack

Stack
Stack is a **virtual environment** + **package development** tool
Main features are:
- **isolation**: **local** to each project i.e. does not affect **global** (aka system) environment
e.g. does not affect **system** GHC installation, if any
- **reproducible** (aka **frozen**) **builds**: allow same **environment** to be reproduced anywhere/anytime
e.g. GHC version, compatible package versions, etc
Implemented using **snapshots** of an environment.

e.g. Comparison

	Package system	**Package management tool**	**Package development tool**	**Virtual Environment**
Python	PyPi package file=setup.py	pip	setup_tools?	virtualenv
Node.js	NPM packages package file= package.json, package-lock.json	npm	npm config=package.json	nvm config=.nvmrc
Haskell	Cabal packages package file = <package>.cabal	cabal-install	Stack config=package.yml	Stack config=stack.yml

Snapshot aka resolver
Snapshot specification: https://docs.haskellstack.org/en/stable/custom_snapshot/
A **snapshot** represents a fixed Haskell **environment**.
A snapshot includes **compatible package versions**
i.e. snapshot **resolves dependencies**. Hence snapshots are also called **resolvers**.

A snapshot consists of:
Ref: https://docs.haskellstack.org/en/stable/custom_snapshot/
- **GHC version**: e.g. ghc-9.2.4
- **GHC options:** e.g. -O2
- **packages** + **compatible versions**: e.g. text-1.2.5.0
- **flags**: ?

Stackage
Stackage: https://www.stackage.org/#about
Standard snapshots are provided by **Stackage**.

Stack vs Cabal vs HPack
stack.yml vs package.yml: https://docs.haskellstack.org/en/stable/stack_yaml_vs_cabal_package_file/
Cabal is the original Haskell **Packaging system**.
e.g. define package name, dependencies, etc
Cabal packages are defined by the ***.cabal** file.

Stack is a **package development tool**.
i.e. tool to facilitate developing **Cabal packages**, especially since Cabal's tools (cabal-install) are old and hard to use.
e.g. cabal-install only handles **global** installs.
HPack is one of the tools used by **Stack**.
HPack autogenerates the hard-to-use **Cabal package file**=*.cabal from a more modern **package.yaml** file

Component
A project consists of mainly 3 types of code:
- **modules** aka **library**
- **executables**
- **tests**
Each is called a **component**.
Components are used as **targets** in a **command**.
e.g.

```
  $ stack build myproject:lib
  $ stack build myproject:exe:myproject-exec
```

Component syntax
In general, components have the form

```
<package name>:<component type>:<name>
```

e.g. components for package=myproject

```
# ./package.yml
name:                myproject
...
library:
  source-dirs: ./src
executables:
 myMain:
    main:                Main.hs
    source-dirs:         ./app
tests:
  myTest:
    main:                Spec.hs
    source-dirs:         ./test
```

Components are

```
myproject:lib
myproject:exe:myMain
myproject:test:myTest
```

Files
Global files:
~/.local/bin/ where project executables are installed by the `stack install` command
~/.stack all stack-related global files
~/.stack/config.yaml Global configuration
~/.stack/snapshots/ Snapshots
~/.stack/snapshots/x86_64-osx/8408abec9... e.g. snapshot
~/.stack/programs/x86_64-osx/ghc-9.2.4/bin/ghc-9.2.4 e.g. GHC location

Project files:
./stack-work Local
./stack.yml Stack config
./package.yml HPack file, used to autogenerate *.cabal file
./<project>.cabal [Autogenerated] Cabal file i.e. package file
./Setup.hs Cabal project file. Not needed for Stack

==========
Configuration (stack.yml)

Ref: https://docs.haskellstack.org/en/stable/yaml_configuration/#resolver-or-snapshot

stack.yaml
Stack's configuration file
i.e. mainly for managing Stack's **virtual environment**.
i.e. in general, NOT for managing the **package definition**.
Package definition is **package.yaml**.

e.g.

```
# stack.yml
resolver: ghc-9.2
packages: []
extra-deps:
- text-1.2.5.0
flags: {}
system-ghc: false
```

```
## resolver
#
# Which snapshot to user
#
# By Stackage snapshot name
#
# Stackage LTS (Long Term Support): https://
www.stackage.org/lts
resolver: lts-19.30                    # LTS version
resolver: https://raw.githubusercontent.com/
commercialhaskell/stackage-snapshots/master/lts/
```

```
19/30.yaml`    # Same

# Stackage Nightly: https://www.stackage.org/nightly
resolver: nightly-2022-10-27           # Nightly version
https://raw.githubusercontent.com/commercialhaskell/
stackage-snapshots/master/nightly/2022/10/27.yaml`     #
Same

# By GHC version
resolver: ghc-9.2.4

# By Custom URL
resolver: https://raw.githubusercontent.com/
commercialhaskell/stackage-snapshots/master/nightly/
2018/8/21.yaml`

# By local path
resolver: my-local-snapshot.yml

# Full specification
resolver:
  url: https://raw.githubusercontent.com/
commercialhaskell/stackage-snapshots/master/lts/
12/0.yaml
  size: 499143
  sha256:
781ea577595dff08b9c8794761ba1321020e3e1ec3297fb833fe951c
ce1bee11


## Local dependencies
#
#
packages:
- .                # Current project
- ./lib/mylib      # ./lib/mylib/package.yml


## Extra dependencies
#
# i.e. dependencies other than packages specified in
snapshot
# Only if not in snapshot or overriding version in
snapshot.
extra-deps:
# - mtl-2.2.2           # No need if already in
snapshot
                        # e.g. mtl-2.2.2 is in https://
www.stackage.org/nightly-2022-10-28
- mtl-2.2.3             # Override

# NOTE: Need to be added to package.yml to use

# ./package.yml
name:                tmp
version:             0.1.0.0
```

```
github:              "githubuser/tmp"
...

dependencies:
- base >= 4.7 && < 5
- text
- mtl < 2.2.3                           # Specified
...
```

```
# Formats
# By Formats
- mtl-2.2.3             # By Hackage version (https://
hackage.haskell.org/package/mtl-2.3)
- mtl                   # ERROR. Need version
-
mtl-2.3@sha256:9850384335d39f4307bdd8010c61b61cc2adb88ad
cb6820f8372f4d2f4905a3d,1830        # With SHA256

# By full specification
- hackage:
mtl-2.3@sha256:9850384335d39f4307bdd8010c61b61cc2adb88ad
cb6820f8372f4d2f4905a3d,1830
  pantry-tree:
    sha256:
0f60bd67fb750f8ce7471fbf8b88b62b75079270c3dc56b7ed59920e
57fa124a
    size: 1852
```

```
# i.e. same as the content in stack.yml

# This file was autogenerated by Stack.
# You should not edit this file by hand.

packages:
- completed:
    hackage:
mtl-2.3@sha256:9850384335d39f4307bdd8010c61b61cc2adb88ad
cb6820f8372f4d2f4905a3d,1830
    pantry-tree:
      sha256:
0f60bd67fb750f8ce7471fbf8b88b62b75079270c3dc56b7ed59920e
57fa124a
      size: 1852
  original:
    hackage: mtl-2.3
...
```

```
# By github commit
- git: https://github.com/bitemyapp/esqueleto.git
  commit: 08c9b4cdf977d5bcd1baba046a007940c1940758
```

```
##
drop-packages:
 -

## Other package options
flags:
- ...
hidden:
- ...



## GHC settings
#
compiler: ghc-9.2.3             # Override compiler
system-ghc: false               # Use system's GHC, not
Stack's
install-ghc: true               # auto-install GHC if
missing
install-ghc: false              # Do not auto-install.
Build usually fails.
ghc-options:
 "$local": -XKindSignatures
 "$everything": ...
```

==========
Cabal file (package.yml, *.cabal)

HPack: https://github.com/sol/hpack
Cabal package file: https://cabal.readthedocs.io/en/stable/cabal-package.html

HPack
package.yaml
HPack is a tool for autogenerating Cabal package files ***.cabal**
HPack **autogenerates** a *.cabal file from a more usable **package.yaml** file.

e.g. For a project name=myproject
Files

```
./app/Main.hs
./src/Lib.hs
./test/Spec.hs
./package.yml
```

Package.yml

```
# package.yml
name:                myproject
version:             0.1.0.0
dependencies:
- base >= 4.7 && < 5
- mtl

ghc-options:
```

```
- -Wall

language: GHC2021

library:
  source-dirs: ./src

executables:
  myproject-exe:
    main:                Main.hs
    source-dirs:         ./app
    dependencies:
    - myproject

tests:
  myproject-test:
    main:                Spec.hs
    source-dirs:         ./test
    dependencies:
    - myproject
```

Auto-generated <project>.cabal file

```
cabal-version: 1.12
name:           myproject
version:        0.1.0.0
license:        BSD3
license-file:   LICENSE
build-type:     Simple
library
  exposed-modules:
      Lib
  other-modules:
      Paths_myproject
  hs-source-dirs:
      src
  ghc-options: -Wall
  build-depends:
      base >=4.7 && <5
    , mtl
  default-language: Haskell2010
executable myproject-exe
  main-is: Main.hs
  other-modules:
      Paths_myproject
  hs-source-dirs:
      app
  ghc-options: -Wall -threaded -rtsopts -with-rtsopts=-N
  build-depends:
      base >=4.7 && <5
    , mtl
    , myproject
  default-language: Haskell2010
test-suite myproject-test
```

```
  type: exitcode-stdio-1.0
  main-is: Spec.hs
  other-modules:
      Paths_myproject
  hs-source-dirs:
      test
  ghc-options: -Wall -threaded -rtsopts -with-rtsopts=-N
  build-depends:
      base >=4.7 && <5
    , mtl
    , myproject
  default-language: Haskell2010
```

```
## Basic fields
#
name:                myProject
version:             0.1.0.0
github:              "ynishiza/myProject"
license:             BSD3
author:              "Yui Nishizawa"
maintainer:          "yui@email.com"
copyright:           "2022 Yui"
extra-source-files:
- README.md
- CHANGELOG.md


## GHC options
#
ghc-options:
- -Wall
- -Wcompat
- "-Wall -Wcompat"        # multiple
- -XKindSignature         # GHC extension. OK, but use
default-extensions instead.

# GHC Language
# Ref: https://downloads.haskell.org/ghc/latest/docs/
users_guide/exts/control.html
# i.e. default extension set
language: GHC2021                # Recommended.
Extensions=BangPatterns, KindSignatures, etc
language: Haskell2010            # Default

# GHC Extension options
default-extensions:
- KindSignatures
- MultiParamTypeClasses


## Dependencies
#
# i.e. other Cabal modules in
Hackage: hackage.haskell.org/
```

```
# Ref: https://github.com/sol/hpack#dependencies
dependencies:
- mtl                   #
- mtl 2.0.3             # Specific version
- mtl 2.0 >=
- mtl < 2.3
- mtl 2.0 >= && <2.3
- name: mtl
  version: "2.0 >= && <2.3"


## Source files
#
# i.e. src, executable, tests
#
# library: the main library source
# i.e. ./src
library:
 source-dirs: .            # current dir. i.e. ./*.hs
files
 source-dirs: ./src        # ./src/*.hs

 # Library specific overrides, if any
 dependencies: []
 ghc-options: []
 ...


# executables:
# i.e. the main program
executables:
 my-exe:
  source-dirs: ./app
  main: main.hs            # ./app/main.hs
  dependencies:
  - myproject              # i.e. its own library
  - mtl                    # Other executable specific
dependencies
  ...

  # Executable specific overrides
  ghc-options: []
  ...

 # multiple
 my-exe2:
  source-dirs: ./app
  main: main2.hs
  ...


# tests: test runners
tests:
 my-test:
  source-dirs: ./test
```

```
  main: Spec.hs            # ./test/Spec.hs
  dependencies:
  - myproject              # i.e. its own library
  - mtl
  ...

  # Test specific overrides
  ghc-options: []
  ...
```

```
============================================================
                           ======
                             CLI

## Basic
#
$ stack --help
$ stack --version
$ stack --verbosity debug ...        #
silent,error,warn,info,debug
$ stack --verbose ...                # debug
$ stack -v ...
$ stack --silent ...
$ stack --color never ...            # no color. Default
$ stack --color auto ...             # always,auto,never
... etc

# Directory
$ stack --stack-root /home/yui/.stack ...    # specify
root explicitly
$ stack --work-dir . ...                     # specify
package explicitly
$ stack --stack-yaml ./stack.yaml ...        # specify
stack.yaml explicitly
...

#
$ stack upgrade                   # upgrade Stack to
latest


## Query
#
# path: location of stack related paths
# e.g. location of ghc
# e.g. location of global .stack
$ stack path                 # all
```

```
...
stack-root: /Volumes/Data/yuinishizawa/stack
project-root: /Volumes/Data/yuinishizawa/Projects/
playground/haskell/tmp
config-location: /Volumes/Data/yuinishizawa/Projects/
playground/haskell/tmp/stack.yaml
...
```

```
compiler-exe: /Volumes/Data/yuinishizawa/stack/programs/
x86_64-osx/ghc-9.2.3/bin/ghc-9.2.3
compiler-bin: /Volumes/Data/yuinishizawa/stack/programs/
x86_64-osx/ghc-9.2.3/bin
compiler-tools-bin: /Volumes/Data/yuinishizawa/stack/
compiler-tools/x86_64-osx/ghc-9.2.3/bin
local-bin: /Users/yuinishizawa/.local/bin
...
```

```
$ stack path --compiler-exe        # location of GHC
$ stack path --stack-root          # ~/.stack
$ stack path --project-root
$ stack path --compiler-exe
$ stack path --program


# ls: list content
$ stack ls snapshots            # All snapshots in
~/.stack/snapshots
```

```
0534f42a5712273474c1347e02efa08c9af2101867ed93857ce9c0fe
801a9441            e.g. ~/.stack/snapshots/x86_64-osx/
0534f42a571...
0809293076b8b7a984a2ece026872600d382ee7a64b18c05cf59d9ad
5969b902            e.g. ~/.stack/snapshots/x86_64-osx/
0809293076b...
...
```

```
$ stack ls tools                # Tools installed
globally
```

```
ghc-9.2.4
ghc-9.2.3
```

```
$ stack ls dependencies         # Dependencies installed
for package
```

```
array 0.5.4.0
base 4.16.3.0
...
```

```
## Initialization
#
$ stack new myproject
$ stack new --bare myproject        # use existing
directory
$ stack new --force ...               # Overwrite
existing files

# stack init: convert Cabal project to Stack project
$ stack init ...
```

```
## Build
#
# stack build: the main command
$ stack build
$ stack build --help
$ stack build --dry-run ...
$ stack build --exec 'echo hello' ...            # Run
command after build
$ stack build --only-dependencies ...
$ stack build --only-snapshot ...
$ stack build --file-watch ...                   # Watch
+ compile
... etc

# build target: i.e. what to compile
# By default, builds:
# - dependencies
# - library
# - executables
$ stack build
$ stack build myproject            # Same

# target by package
$ stack build myproject            # i.e. this package
$ stack build mtl                  # dependency
$ stack build mtl-2.2.2            # version

# target by component
$ stack build myproject
$ stack build myproject:lib          # library
$ stack build myproject:exe:mymain   # executable


# GHC options
$ stack build --ghc-options="-Wall -Werror" ...
# GHC override
$ stack build --pedantic ...
 # --ghc-options='-Wall -Werror'
$ stack build --fast ...
 # --ghc-options='-O0' i.e. no optimizations
$ stack build --trace ...
$ stack build --profile ...


# test: by default, tests are not compiled
$ stack build --test ...                 # compile + run
tests
$ stack test ...
$ stack build --test --no-run-test

# install (copy-bins): install project on machine
$ stack build --copy-bins ...           # install in
~/.local/bin
```

```
$ stack install ...

# haddock: compile haddock
$ stack build --haddock ...
$ stack haddock ...

# benchmark: ?
$ stack build --bench ...
$ stack bench ...


# stack clean: delete caches in ./.stack-work
#
$ stack clean               #
$ stack clean myproject     # Remove by package
$ stack clean --full         # Remove ./.stack-work
$ stack purge               # same



## Execution
#
# stack exec: execute shell command in a special
environment
# - GHC in PATH
# - package executables in PATH
#
# i.e. similar to Node.js's npx
$ stack exec ls             # Any shell command
$ stack exec ls -- -lt       # with arguments
$ stack exec bash           # Enter shell

# GHC
$ stack exec ghc               # Run ghc
$ stack exec -- ghc             # Same
$ stack ghc
$ stack runghc
$ stack exec -- which ghc       # Stack shell with GHC

e.g.
/home/yui/.stack/programs/x86_64-osx/ghc-9.2.4/bin/ghc

# GHCI
$ stack exec ghci              # OK. Raw GHCI i.e.
packages not loaded
$ stack repl                   # BETTER. Use repl
instead to preload packages (See below)

# Package exec: i.e. run exec built by Stack
$ stack exec -- myMain
$ stack exec -- which myMain    # Path

e.g.
/home/yui/myproject/.stack-work/install/x86_64-osx/
aa323cc34c9333ca25b6f245f9501d5e9af7413e514f74082285334e
```

```
75d8952a/9.2.4/bin/myMain
```

```
# stack repl: GHC REPL, but with the package loaded in
the REPL
#
$ stack repl
$ stack ghci
$ stack exec ghci                 # Same but raw i.e.
nothing loaded

# By component
$ stack repl myproject            # Whole
$ stack repl myproject:lib        # Library i.e. local
modules



## Misc
$ stack sdist
$ stack dot
$ stack eval
$ stack script
... etc
```